UNIVERSIDAD DEL ISTMO

Campus Tehuantepec

Ingeniería en Computación

Tema: **Proyecto para evaluación ordinaria.**

COMPILADOR E INTÉRPRETE PARA LA EVALUACIÓN DE MÉTODOS NUMÉRICOS.

Materia: **Compiladores.**

Alumnos: **Joan Pablo Alvarado Garfias.**

Jeremy Osorio Ramos.

Docente: **Dr. J. Jesús Arellano Pimentel.**

Semestre: **Quinto.**

Grupo: ***504.***

Tehuantepec Oaxaca a 06 de febrero de 2025.

ÍNDICE

1. Introducción

2. Método de Newton

3. Método de punto fijo

4. Gramática

5. Ejemplos de prueba

6. Conclusión

INTRODUCCIÓN

El desarrollo de herramientas computacionales especializadas en el cálculo numérico es fundamental en diversas áreas de la ciencia e ingeniería. En este contexto, se presenta un compilador e intérprete diseñado para la evaluación de ecuaciones algebraicas mediante los métodos numéricos de Newton y punto fijo. Este sistema permite la entrada de ecuaciones en un formato especificado, validando su sintaxis y realizando la evaluación según el método seleccionado.

El objetivo principal de este trabajo es el desarrollo de un compilador e intérprete capaz de aceptar como entrada ecuaciones o sistemas de ecuaciones algebraicas que puedan ser resueltas por métodos numéricos específicos. Para lograrlo, se han establecido distintos objetivos específicos, entre los cuales se encuentran el diseño de la especificación léxica y sintáctica, la implementación de acciones semánticas, el desarrollo de una interfaz integrada y la incorporación de mecanismos para la detección de errores sintácticos.

MÉTODO DE NEWTON

El Método de Newton-Raphson es una técnica iterativa que se utiliza para encontrar raíces de funciones no lineales. En otras palabras, sirve para resolver ecuaciones de la forma: $\mathbf{f(x)=0}$.
Suponga que se tiene una curva (la gráfica de una función) y necesita encontrar el punto donde corta el eje x (es decir, su raíz). El Método de Newton utiliza una aproximación inicial (un valor que cree que está cerca de la raíz) y, a partir de ahí, mejora esa aproximación paso a paso.

1. Empieza con un valor inicial x_0 (EL primer intento de adivinar la raíz).
2. Traza la recta tangente a la función en ese punto.
3. Ve dónde esa tangente cruza el eje x.
4. Ese punto de cruce es la nueva aproximación x_1
5. Repite el proceso hasta que esté lo suficientemente cerca de la raíz real.

La fórmula se presenta a continuación:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Donde:

$\boldsymbol{x_n}$ es la aproximación actual.

$\boldsymbol{f(x_n)}$ es el valor de la función en x_n.

$\boldsymbol{f'(x_n)}$ es la derivada de la función evaluada en x_n.

$\boldsymbol{x_{n+1}}$ es la nueva aproximación.

MÉTODO DE PUNTO FIJO

El Método de Punto Fijo es una técnica iterativa que se utiliza para encontrar soluciones de ecuaciones de la forma: $\mathbf{f(x)=0}$

Sin embargo, en lugar de resolver directamente esta ecuación, se reescribe de la siguiente forma: $\mathbf{x=g(x)}$.

Aquí, se busca un valor de x que no cambie al aplicar la función $\mathbf{g(x)}$. A este valor se le llama punto fijo.

La fórmula se presenta a continuación:

$$x_{n+1} = g(x_n)$$

Donde:

$(\boldsymbol{x_n})$ es la aproximación actual.

$\boldsymbol{g(x_n)}$ es la nueva aproximación.

Un punto fijo es un valor x que satisface la condición:

$$x = g(x)$$

Si se aplica la función g una y otra vez, eventualmente se llega a un valor que ya no cambia. Ese es el punto fijo, que corresponde a la raíz de la ecuación original $\mathbf{f}(\mathbf{x}) = \mathbf{0}$.

GRAMÁTICA

Este proyecto implementa un analizador sintáctico y un analizador léxico utilizando Bison y Flex para resolver ecuaciones no lineales mediante dos métodos numéricos: Método de Newton-Raphson y Método de Punto Fijo. El código está diseñado para leer una entrada desde un archivo de texto (entrada.txt), procesarla, y luego escribir los resultados en otro archivo de texto (salida.txt).

Estructura del Código

El código está dividido en varias secciones principales:

1. Definiciones y Declaraciones:

Se incluyen las bibliotecas necesarias y se definen constantes para los métodos numéricos (METODO_NEWTONRAPH y METODO_PUNTOFIJO).

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <string.h>
#include "parsermet.h"

#define EVALUAR_FUNCION 997
#define METODO_NEWTONRAPH 998
#define METODO_PUNTOFIJO 999
#define EPSILON 1e-9

#ifndef M_PI
    #define M_PI 3.14159265358979323846
#endif

#ifndef M_E
    #define M_E 2.71828182845904523536
#endif
```

Imagen 1.0 (Librerías y constantes necesarias).

Se declaran variables globales como *x*, *fx*, *fdx*, *gx*, y *error_esperado*, que se utilizan para almacenar los valores durante el proceso de iteración.

Se definen los tokens y las reglas gramaticales para el analizador sintáctico.

```
double x = 10.0;
double error_esperado = 0.1;
int metodo_sel = 0;

int set_x = 0;

double fx = 0.0, fdx = 0.0, gx = 0.0;

%}

%union {
    double val;
}

%type <val> expr term factor sig
%token <val> NUMERO
%token X EULER PI
%token SIN COS TAN SEC COSEC COT ASIN ACOS ATAN SINH COSH TANH
%token LOG10 LOGE EXP RAIZ

%token NT_DEC PF_DEC X_DEC FX_DEC FDX_DEC GX_DEC ERROR_DEC
%token ERROR_T

%left '+' '-'
%left '*' '/'
%right '^'
```

. Imagen 1.2 (Tokens y constantes necesarias).

2. Reglas Gramaticales:

Las reglas gramaticales definen cómo se deben interpretar las expresiones matemáticas. Estas reglas incluyen operaciones básicas como suma, resta, multiplicación, división, y funciones matemáticas como seno, coseno, tangente, logaritmo y raíz cuadrada.

Las reglas también manejan la asignación de valores a las variables *x*, *fx, fdx*, y *gx* dependiendo del método seleccionado.

```
prog:
    /*
    Gramatica para metodo de Newton Raphson
    P -> nt{
            x = NUMERO
            f(x) = expr
            fd(x) = expr
            error = NUMERO
        }
    Gramatica para metodo de Punto Fijo
    P -> pf{
            x = NUMERO
            f(x) = expr
            g(x) = expr
            error = NUMERO
        }
    Gramatica para evaluar funciones
    P -> {
        x = NUMERO
        f(x) = expr
        }

    Gramatica de expr
    expr ->   expr + term
            | expr - term
            | term

    Gramatica de term
    term ->   term * factor
            | term / factor
            | term ^ factor
            | factor

    Gramatica de factor
    factor ->   NUMERO
                | X
                | EULER
                | PI
                | SIN
                | COS
                | TAN
                | SEC
                | COSEC
                | COT
                | ASIN
                | ACOS
                | ATAN
                | SINH
                | COSH
                | TANH
                | LOG10
                | LOGE
                | EXP
                | RAIZ
                | ( expr )
```

. Imagen 2.0 (Reglas gramaticales).

3. Funciones de Asignación y Error:

asignarValores: Esta función asigna los valores iniciales para *x*, *fx*, *fdx*, *gx*, y *error_esperado* dependiendo del método seleccionado.

```
void asignarValores(double x_metodo, double valor_f1, double valor_f2, double error_metodo){
    if(!set_x){
        x = x_metodo;
        error_esperado = error_metodo;
        set_x = 1;
        return;
    }

    if(metodo_sel == METODO_NEWTONRAPH ||  metodo_sel == EVALUAR_FUNCION){
        fx = valor_f1;
        fdx = valor_f2;
    }
    else if(metodo_sel == METODO_PUNTOFIJO){
        fx = valor_f1;
        gx = valor_f2;
    }
    else{
        yyerror("Metodo no reconocido");
    }
}
```

. Imagen 3.0 (Función agregar valor).

yyerror y yywarning: Estas funciones manejan los errores y advertencias durante el análisis sintáctico.

```
void yyerror(const char *s) {
    fprintf(stderr, "Error (Linea %d): %s\n", yylineno, s);
}

void yywarning(const char *msg) {
    FILE *wn = fopen("advertencias.txt","a+");
    fprintf(wn, "Advertencia (Linea %d): %s\n", yylineno, msg);
    fclose(wn);
}
```

Imagen 3.1 (Funciones yyerror y yywarning).

4. Función Principal (main)

La función principal lee la entrada desde *archivo.txt*, realiza el análisis sintáctico, y luego ejecuta el método numérico seleccionado (Newton-Raphson o Punto Fijo) para encontrar la raíz de la ecuación.

El proceso se repite hasta que el error sea menor que el error esperado.

Los resultados de cada iteración se imprimen en la consola.

```c
int parser() {
    int i = 0;
    double error = 10.0;
    double x_nuevo;

    FILE *res = fopen("resultados.txt", "w");
    fclose(res);
    yyin = fopen("entrada.txt", "r");
    yyout = fopen("salida.txt", "w");
    if(yyparse() != 0){
        yyrestart(yyin);
        yyclearin;
        yylineno = 1;
        fclose(yyin);
        fclose(yyout);
        return 0;
    }
    fclose(yyin);
    fclose(yyout);

    if (metodo_sel != EVALUAR_FUNCION) {
        FILE *res = fopen("resultados.txt", "a+");
        fprintf(res, "%-10s %-12s %-12s", "Iteración", "X", "f(X)");
        if (metodo_sel == METODO_NEWTONRAPH)
            fprintf(res, " %-12s", "f'(X)");
        else
            fprintf(res, " %-12s", "g(X)");
        fprintf(res, " %-12s\n", "Error");

        // Separador visual
        fprintf(res, "---------- ------------ ------------");
        if (metodo_sel == METODO_NEWTONRAPH)
            fprintf(res, " ------------");
        else
            fprintf(res, " ------------");
        fprintf(res, " ------------\n");

        // Cerrar el archivo temporalmente
```

Imagen 4.0 (Función principal).

```
// Cerrar el archivo temporalmente
fclose(res);

// Iterar hasta que el error sea menor que el error esperado
while (error > error_esperado) {
    i++;

    yyin = fopen("entrada.txt", "r");
    if(yyparse() != 0){
        yyrestart(yyin);
        yyclearin;
        yylineno = 1;
        fclose(yyin);
        fclose(yyout);
        return 0;
    }
    fclose(yyin);

    if (metodo_sel == METODO_NEWTONRAPH) {
        validar_denominador(fdx);
        x_nuevo = x - (fx / fdx);
    } else {
        x_nuevo = gx;
    }

    error = fabs(x_nuevo - x);
    x = x_nuevo;

    res = fopen("resultados.txt", "a");

    // Escribir los resultados en formato tabular
    fprintf(res, "%-10d %-12.6f %-12.6f", i, x, fx);
    if (metodo_sel == METODO_NEWTONRAPH)
        fprintf(res, " %-12.6f", fdx);
    else
        fprintf(res, " %-12.6f", gx);
    fprintf(res, " %-12.6f\n", error);
```

Imagen 4.1 (Continuación función principal).

```
            fprintf(res, " %-12.6f\n", error);

            fclose(res);
        }
    } else {
        FILE *res = fopen("resultados.txt", "w");
        yyin = fopen("entrada.txt", "r");
        yyout = fopen("salida.txt", "w");
        if(yyparse() != 0){
            yyrestart(yyin);
            yyclearin;
            yylineno = 1;
            fclose(yyin);
            fclose(yyout);
            return 0;
        }

        fprintf(res, "f(%.6f) = %.6f\n", x, fx);

        fclose(yyin);
        fclose(yyout);
        fclose(res);
    }

    x = 10.0;
    metodo_sel = 0, set_x = 0, i = 0;
    fx = 0.0, fdx = 0.0, gx = 0.0, error_esperado = 0.2;
    yylineno = 1;
    return 1;
}
```

Imagen 4.2 (Fin función principal).

Compilación

Para compilar el código, es de vital importancia asegurarse de tener instalados Bison y Flex. En caso de no tenerlo puede visitar el sitio oficial en los siguientes enlaces y descargarlo de manera gratuita: https://www.gnu.org/software/bison/, https://gothub.projectsegfau.lt/westes/flex. Posteriormente, se ejecutan los siguientes comandos en la terminal:

```
bison -d newtonrap.y

flex newtonrap.l

gcc newtonrap.tab.c lex.yy.c -lm -o newtonrap
```

Estos comandos generan 3 archivos de cabecera. Para generar un archivo *.h* y un archivo *.c* para realizar pruebas en consola, se necesita ejecutar:

```
flex.exe –oparsermet.h parsermet.l
```

```
bison.exe -d –oparsermet_yac.c parsermet.y
```

De igual manera de necesita crear un archivo llamado entrada.txt en el mismo directorio que el ejecutable. El archivo debe contener la definición de la ecuación y los parámetros iniciales. Por ejemplo:

```
nt{
x = -2.0
f(x) = x^2 - 2
fd(x) = 2*x
error = 0.01
}
```

Esto indica que se utilizará el método de Newton-Raphson con un valor inicial de x = -2.0, la función: $\mathrm{f(x) = x2 - 2}$, su derivada: $\mathrm{f'(x) = 2 * x}$ y un error esperado de 0.01.

Finalmente se ejecuta el programa compilado. Los resultados se imprimirán en un archivo llamado resultados.txt.

Ejemplo de salida:

Iteración	X	f(X)	g(X)	Error
1	0.400000	2.000000	0.400000	0.400000
2	0.469404	0.347022	0.469404	0.069404
3	0.498600	0.145980	0.498600	0.029196
4	0.512893	0.071462	0.512893	0.014292
5	0.520361	0.037343	0.520361	0.007469

Consideraciones

El código está diseñado para cumplir algunas consideraciones como son:

División por Cero: El código maneja la división por cero lanzando un error.

Precisión: El algoritmo se detiene cuando el error es menor que el error esperado.

Reinicio: Después de cada ejecución, las variables globales se reinician a sus valores iniciales.

EJEMPLOS DE PRUEBAS

La siguiente imagen es una vista de la interfaz gráfica desarrollada por los integrantes del equipo la cual cuenta con los siguientes incisos.

a) Botones de funcionalidad básica.
   Donde **nuevo** limpia el contenido en pantalla para comenzar una nueva evaluación.
   Abrir permite acceder al explorador de archivos y abrir un archivo de texto que contenga una gramática por validar.
   Guardar permite guardar la gramática evaluada en un archivo de texto.
   Compilar compila, analiza y evalúa el texto ingresado para comprobar si la sintaxis ingresada es correcta o no y mostrar un resultado.
   Acerca de muestra información acerca del software, como los desarrolladores y la versión.
   Salir cierra la ventana y sale de la aplicación.
b) Área de edición de texto.
   Permite ingresar texto para ser evaluado.
c) Área de notificación de errores y advertencias.
   Permite visualizar los errores o advertencias (si existen) obtenidos después de analizar la gramática.
d) Área de resultados.
   Permite visualizar los datos de la evaluación.

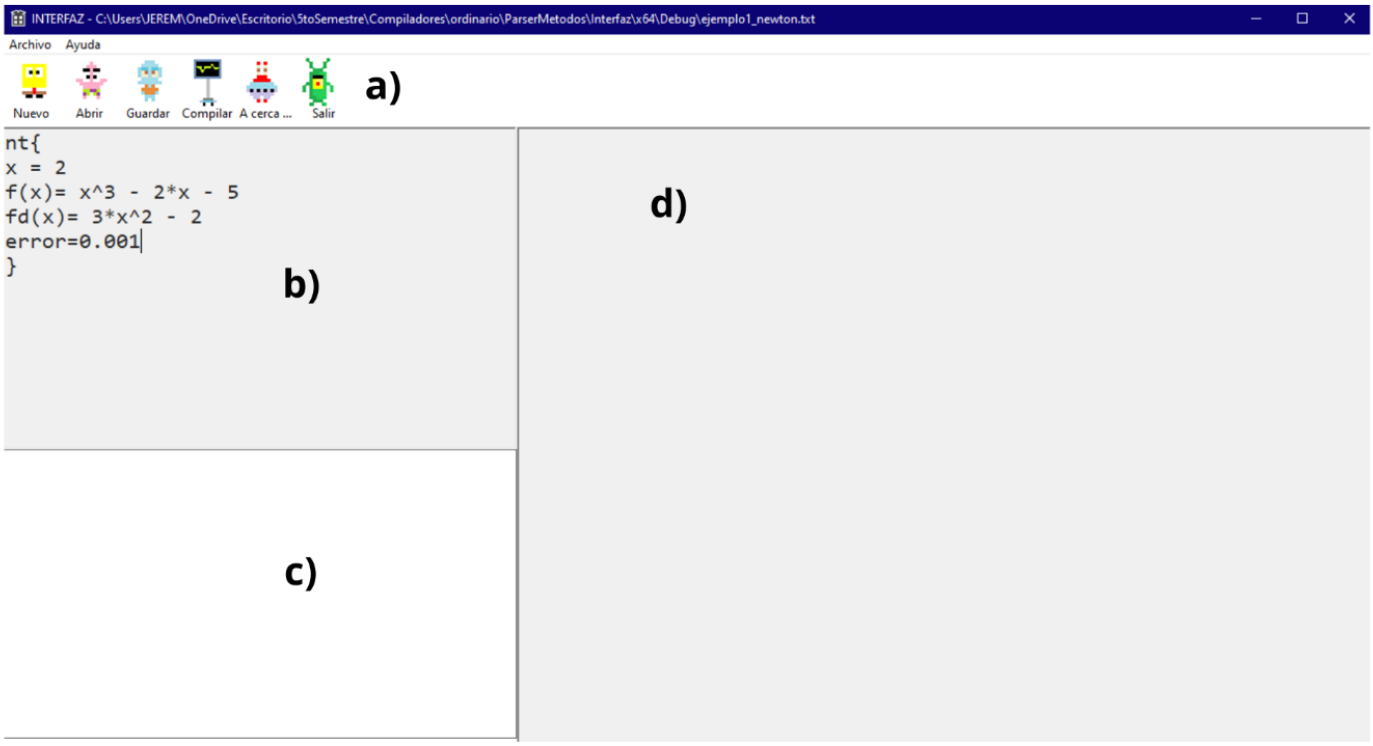

Imagen 5.0 (vista principal).

Los siguientes ejemplos se implementaron en la interfaz gráfica desarrollada por los integrantes del equipo, la cual contiene la lógica y gramática antes mencionada además de funcionalidades como abrir, guardar, ejecutar o crear un nuevo archivo de texto para posteriormente analizarlo y comprobar que cumpla con la gramática establecida para resolver uno de los dos métodos numéricos desarrollados.

Ejemplo 1, método de newton:

Para el siguiente ejemplo la sintaxis a ingresar es la siguiente:

```
nt{
x = 2
f(x)= x^3 - 2*x - 5
fd(x)= 3*x^2 - 2
error=0.001
}
```

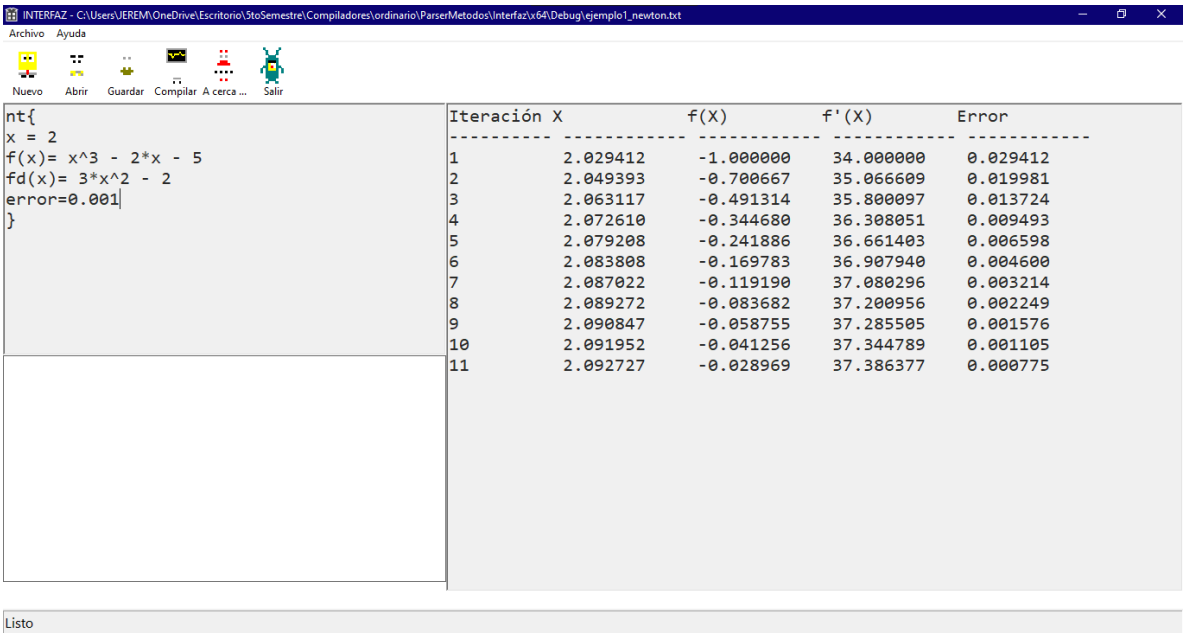

Imagen 6.0 (Resultados de ejemplo 1).

Ejemplo 2, punto fijo:

Para el siguiente ejemplo la sintaxis a ingresar es la siguiente:

```
pf{
x = 0
f(x) = 2*exp(x^2) - 5*x
g(x) = 0.4*exp(x^2)
error = 0.01
}
```

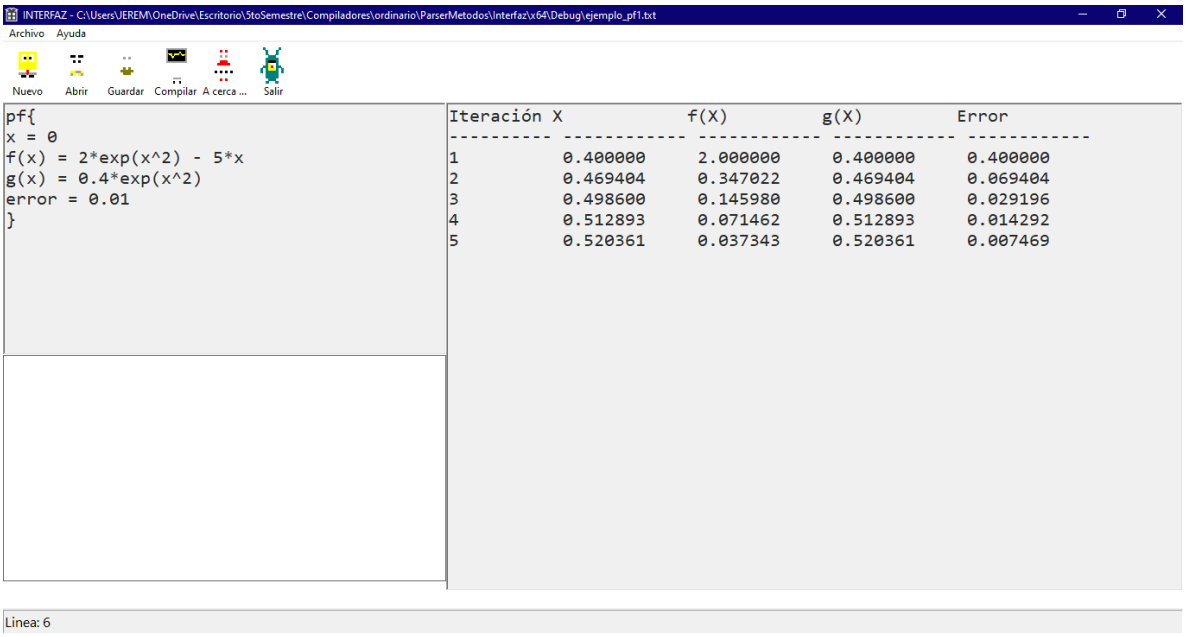

Imagen 6.1 (Resultados de ejemplo 2).

Puede comprobar los resultados del siguiente video de youtube:

https://youtu.be/8b75oripNyw?si=Q4phDSFtqBbuPqv_

CONCLUSIÓN

Este proyecto ha sido una experiencia retadora que nos ha permitido fortalecer habilidades en el diseño y desarrollo de software orientado a la resolución de problemas numéricos. La creación del compilador e intérprete nos ha llevado a comprender más a fondo la estructura de los lenguajes de programación y la importancia de la validación sintáctica y semántica en los sistemas computacionales. Además, la implementación de mecanismos para la detección y notificación de errores ha mejorado la usabilidad del sistema, facilitando la corrección de posibles fallos en las ecuaciones ingresadas. En el futuro, podría ampliarse este trabajo incorporando nuevos métodos numéricos y optimizando el rendimiento del compilador para ecuaciones de mayor complejidad.

BIBLIOGRAFÍA

Nieves A. (2006). Métodos numéricos Aplicados a la ingeniería. Editorial Continental.